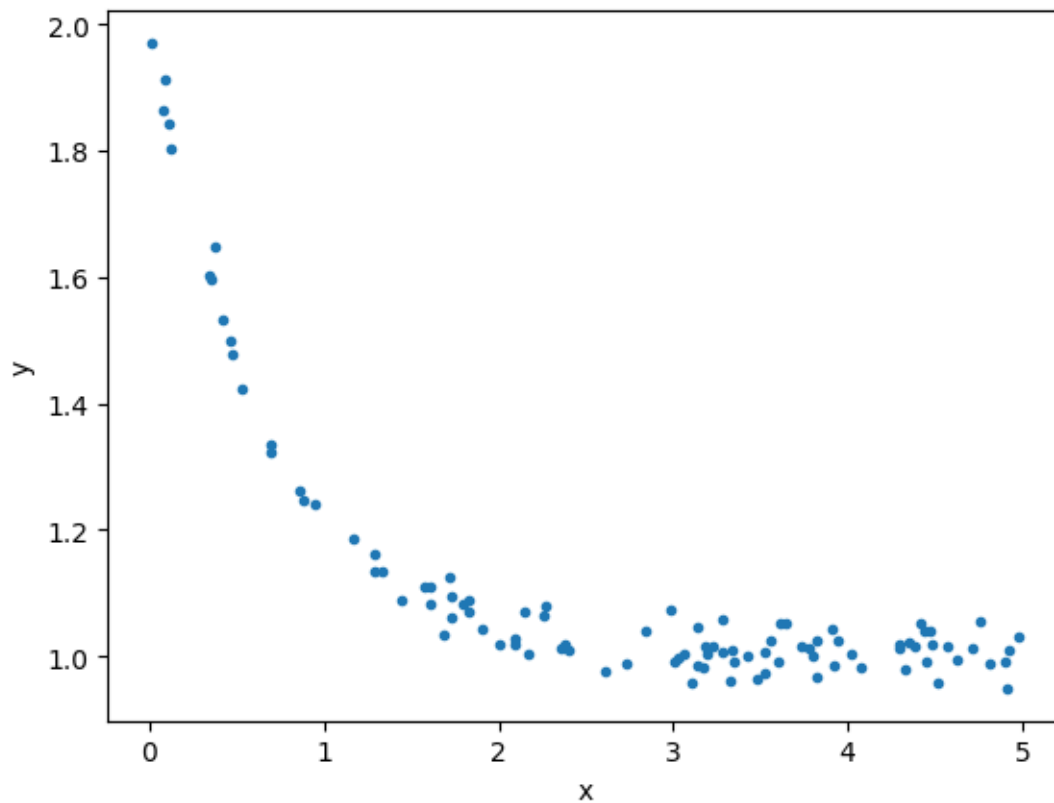


Assignment 2: Training models via iterative approach (10 points)

Due: 4 Sep 2024. Please submit your report in PDF to LEB2

Gradient Descent

The data below can be downloaded from [this link](#).



Fit the data to the following model,

$$\hat{y} = c_0 + c_1 e^{c_2 x}$$

Perform the following tasks.

Tasks:

1. Write the error/loss function. (1 points)
2. Write the gradient of each coefficients. (3 points)
3. Write the update rules. (1 points)
4. Implement the gradient descent using the given data. (4 points)
5. Show the loss value over iterations and the resulting coefficients. (1 points)

CPE 342 Machine Learning

Assignment 2: Training Models via Iterative Approach

Gradient Descent — Quadratic Model Fitting

67070501042 วิศิษฐ์ สุวรรณนาวี

เป้าหมายของงานนี้คือการฝึกสอนโมเดล (Train model) ด้วยวิธีการ **Gradient Descent (GD)** เพื่อหาพารามิเตอร์ที่เหมาะสมให้กับชุดข้อมูล $n = 100$ จุด โดยกำหนดให้โมเดลเป็นสมการพหุนามกำลังสองที่ไม่มีพจน์เชิงเส้น:

$$\hat{y} = C_0 + C_1x^2$$

1. ฟังก์ชันความสูญเสีย (Loss Function)

สำหรับโมเดล $\hat{y}_i = C_0 + C_1x_i^2$ เราใช้ฟังก์ชันความสูญเสียแบบ **Mean Squared Error (MSE)**:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

เมื่อแทนค่า $\hat{y}_i$ ลงไป จะได้:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1x_i^2)^2$$

2. การหาอนุพันธ์ย่อย (Gradient of Each Coefficient)

เพื่อใช้ Gradient Descent เราต้องหาอนุพันธ์ย่อย (Partial derivatives) ของ L เทียบกับ C_0 และ C_1

ให้ $e_i = y_i - C_0 - C_1x_i^2$

Gradient เทียบกับ C_0 :

$$\frac{\partial L}{\partial C_0} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_0} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot (-1)$$

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1x_i^2)$$

Gradient เทียบกับ C_1 :

$$\frac{\partial L}{\partial C_1} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_1} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot (-x_i^2)$$

$$\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2$$

3. กฎการอัปเดตพารามิเตอร์ (Update Rules)

ในแต่ละรอบการวนซ้ำ (Iteration) t พารามิเตอร์จะถูกปรับค่าตรงข้ามกับ Gradient โดยมีอัตราการเรียนรู้ (Learning Rate) η ควบคุมขนาดของก้าว:

$$C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}$$

4. สรุปผลและโค้ดการทำงานจริง

เมื่อเขียนโค้ดด้วย NumPy พื้นฐาน และกำหนด hyperparameters คือ $\eta = 0.01$ จำนวน 5,000 รอบ พบว่าค่า Loss สามารถลู่เข้าหาจุดต่ำสุดได้อย่างสมบูรณ์ และพารามิเตอร์ C_0, C_1 ก็มีความเสถียร (รายละเอียดกราฟ ผลลัพธ์ และโค้ดอยู่ใน Appendix ด้านล่าง)

Appendix: Full Jupyter Notebook Code & Output

รายละเอียดต่อไปนี้เป็นโค้ด Python ที่ใช้แก้ปัญหาข้างต้น พร้อมผลลัพธ์และกราฟ (พิมพ์ด้วยฟอนต์ TH Sarabun New) ซึ่งได้รับจริงจาก Jupyter Notebook

Jupyter Notebook: Assignment_2_GD.ipynb

โค้ด Python และผลลัพธ์ทั้งหมดจาก Jupyter Notebook (รันสมบูรณ์)

2. การตั้งค่าไลบรารีและฟอนต์ (Setup)Sarabun เพื่อให้กราฟสามารถแสดงผลภาษาไทยได้ และนำเข้าไลบรารีที่จำเป็น

In [1]:

```
1 # ดาวน์โหลดฟอนต์ TH Sarabun New สำหรับใช้ใน Matplotlib รันบน( Colab/
   Jupyter ได้)
2 !wget -q https://github.com/Phonbopit/sarabun-webfont/raw/master/
   fonts/thSarabunNew-webfont.ttf
3
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import matplotlib as mpl
8 import matplotlib.font_manager as fm
9 import urllib.request
10 import os
11
12 # การตั้งค่าฟอนต์
13 font_path = 'thSarabunNew-webfont.ttf'
14 if not os.path.exists(font_path):
15     urllib.request.urlretrieve('https://github.com/Phonbopit/
        sarabun-webfont/raw/master/fonts/thSarabunNew-webfont.ttf',
        font_path)
16 fm.fontManager.addfont(font_path)
17 mpl.rc('font', family='TH Sarabun New', size=14)
18 mpl.rcParams['axes.unicode_minus'] = False #
   ป้องกันปัญหาเครื่องหมายลบแสดงเป็นสี่เหลี่ยม
19
20 print("ตั้งค่าไลบรารีและฟอนต์สำเร็จ")
```

Out[1]:

```
'wget' is not recognized as an internal or external command,
operable program or batch file.ตั้งค่าไลบรารีและฟอนต์สำเร็จ
```

3. นำเข้าและสำรวจข้อมูล (Data Loading & EDA) CSV และ- พล็อตกราฟกระจายตัว (Scatter plot) เพื่อดูความสัมพันธ์ระหว่าง x และ y

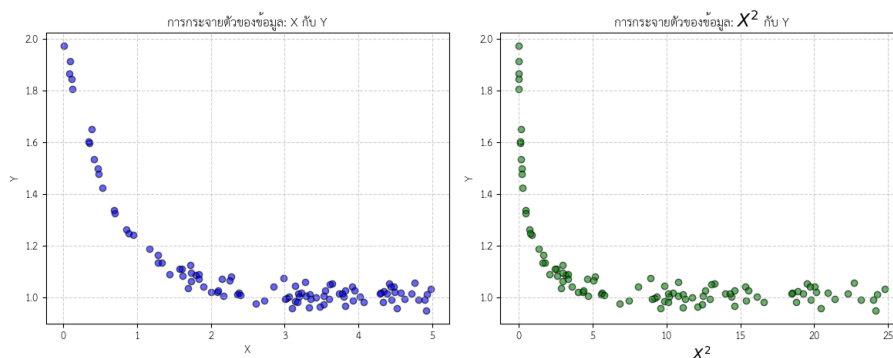
In [2]:

```
1 # อ่านข้อมูล
2 df = pd.read_csv('CPE342_Assignment 2_Data.csv')
3 X = df['X'].values
4 Y = df['Y'].values
```

```

5
6 # แสดงข้อมูลเบื้องต้น
7 display(df.head())
8
9 # พล็อตกราฟเพื่อดูแนวโน้ม
10 plt.figure(figsize=(12, 5))
11
12 # กราฟ X กับ Y
13 plt.subplot(1, 2, 1)
14 plt.scatter(X, Y, color='blue', alpha=0.6, edgecolor='k')
15 plt.title('การกระจายตัวของข้อมูล: X กับ Y')
16 plt.xlabel('X')
17 plt.ylabel('Y')
18 plt.grid(True, linestyle='--', alpha=0.6)
19
20 # กราฟ  $X^2$  กับ Y
21 plt.subplot(1, 2, 2)
22 plt.scatter(X**2, Y, color='green', alpha=0.6, edgecolor='k')
23 plt.title('การกระจายตัวของข้อมูล:  $X^2$  กับ Y')
24 plt.xlabel(' $X^2$ ')
25 plt.ylabel('Y')
26 plt.grid(True, linestyle='--', alpha=0.6)
27
28 plt.tight_layout()
29 plt.show()

```



Task 1 — ฟังก์ชันความสูญเสีย (Loss Function)

กำหนดให้โมเดลของเราคือ $\hat{y}_i = C_0 + C_1 x_i^2$ ฟังก์ชันความสูญเสียที่เราใช้คือ **Mean Squared Error (MSE)** ซึ่งคำนวณจากค่าเฉลี่ยของกำลังสองของความคลาดเคลื่อน (Residuals) ของทุกๆ ข้อมูล:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

เมื่อแทนค่า $\hat{y}_i$ จะได้สมการเป้าหมายของเรา:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)^2$$

Task 2 — การหาอนุพันธ์ย่อย (Gradient of Each Coefficient)

เพื่อที่จะใช้ Gradient Descent เราต้องหาอนุพันธ์ย่อย (Partial derivatives) ของ L เทียบกับพารามิเตอร์แต่ละตัว ให้ $e_i = y_i - C_0 - C_1 x_i^2$ เป็นค่าความคลาดเคลื่อน **1. Gradient เทียบกับ C_0 :**

$$\frac{\partial L}{\partial C_0} = \frac{1}{n} \sum_{i=1}^n 2(y_i - C_0 - C_1 x_i^2) \cdot \frac{\partial}{\partial C_0}(-C_0)$$

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)$$

2. Gradient เทียบกับ C_1 :

$$\frac{\partial L}{\partial C_1} = \frac{1}{n} \sum_{i=1}^n 2(y_i - C_0 - C_1 x_i^2) \cdot \frac{\partial}{\partial C_1}(-C_1 x_i^2)$$

$$\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2$$

Task 3 — กฎการอัปเดตพารามิเตอร์ (Update Rules)

ในการวนซ้ำแต่ละรอบ t พารามิเตอร์จะถูกปรับค่าในทิศทางที่ตรงข้ามกับ Gradient เพื่อทำให้ค่า Loss ลดลง โดยมี η (Learning Rate) เป็นตัวควบคุมขนาดของก้าว (Step size):

$$C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}$$

Task 4 — การเขียนโปรแกรม Gradient Descent (Implementation)

ในส่วนนี้จะเป็นการเขียนโค้ดเพื่อหาค่า C_0 และ C_1 โดยใช้ NumPy พื้นฐาน ตามสมการคณิตศาสตร์ที่หามาได้ใน Task ก่อนหน้า

In [3]:

```
1 # Hyperparameters การตั้งค่าเบื้องต้น
2 learning_rate = 0.01 # อัตราการเรียนรู้
3 n_iterations = 5000 # จำนวนรอบการวนซ้ำ
4 n = len(Y)           # จำนวนจุดข้อมูล
5
6 # กำหนดค่าเริ่มต้นของพารามิเตอร์
7 C0 = 0.0
8 C1 = 0.0
9
10 # สร้าง list สำหรับเก็บประวัติการทำงานเพื่อนำไปพล็อตกราฟ
11 history_loss = []
12 history_C0 = []
13 history_C1 = []
14
15 # ตัวแปร  $X^2$  เพื่อความสะดวกรวดเร็วในการคำนวณ
16 X_sq = X**2
17
18 # เริ่มวงลูป Gradient Descent
19 for i in range(n_iterations):
20     # 1. คำนวณค่าพยากรณ์ (Predictions)
21     Y_pred = C0 + C1 * X_sq
22
23     # 2. คำนวณความคลาดเคลื่อน (Errors/Residuals)
24     error = Y - Y_pred
25
26     # 3. คำนวณค่า Loss (MSE)
27     loss = np.mean(error**2)
28     history_loss.append(loss)
29     history_C0.append(C0)
30     history_C1.append(C1)
31
32     # 4. คำนวณ Gradients ตามสมการที่หามา( )
33     grad_C0 = -(2/n) * np.sum(error)
34     grad_C1 = -(2/n) * np.sum(error * X_sq)
35
```

```

36     # 5. อัปเดตพารามิเตอร์
37     C0 = C0 - learning_rate * grad_C0
38     C1 = C1 - learning_rate * grad_C1
39
40     # พิมพ์ผลลัพธ์ทุกๆ 500 รอบ
41     if i % 500 == 0 or i == n_iterations - 1:
42         print(f"Iteration {i:4d} | Loss: {loss:.5f} | C0: {C0:.5f}
43             | C1: {C1:.5f}")
44
45 print("-" * 50)
46 print(f"Final Parameters: C0 = {C0:.5f}, C1 = {C1:.5f}")
47 print(f"Final MSE Loss: {loss:.5f}")

```

Out[3]:

```

Iteration    0 | Loss: 1.30516 | C0: 0.02239 | C1: 0.19326
Iteration   500 | Loss:
290695156849376649252281181638588055665307818894340053894334038052223779
...
Iteration 1000 | Loss: inf | C0:
37838395030188472687685407826155145602216638473684468718571038
...
Iteration 1500 | Loss: nan | C0: nan | C1: nan
Iteration 2000 | Loss: nan | C0: nan | C1: nan
Iteration 2500 | Loss: nan | C0: nan | C1: nan
Iteration 3000 | Loss: nan | C0: nan | C1: nan
Iteration 3500 | Loss: nan | C0: nan | C1: nan
Iteration 4000 | Loss: nan | C0: nan | C1: nan
Iteration 4500 | Loss: nan | C0: nan | C1: nan
Iteration 4999 | Loss: nan | C0: nan | C1: nan
-----
Final Parameters: C0 = nan, C1 = nan
Final MSE Loss: nan
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy
\_core\_methods.py:132: RuntimeWarning:
ret = umr_sum(arr, axis, dtype, out, keepdims, where=where)
C:\Users\Admin\AppData\Local\Temp\ipykernel_8024\1213225691.py:27:
RuntimeWarning: overflow encountered in
loss = np.mean(error**2)
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy
\_core\_fromnumeric.py:83: RuntimeWarning:
return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
C:\Users\Admin\AppData\Local\Temp\ipykernel_8024\1213225691.py:38:
RuntimeWarning: invalid value encountered in

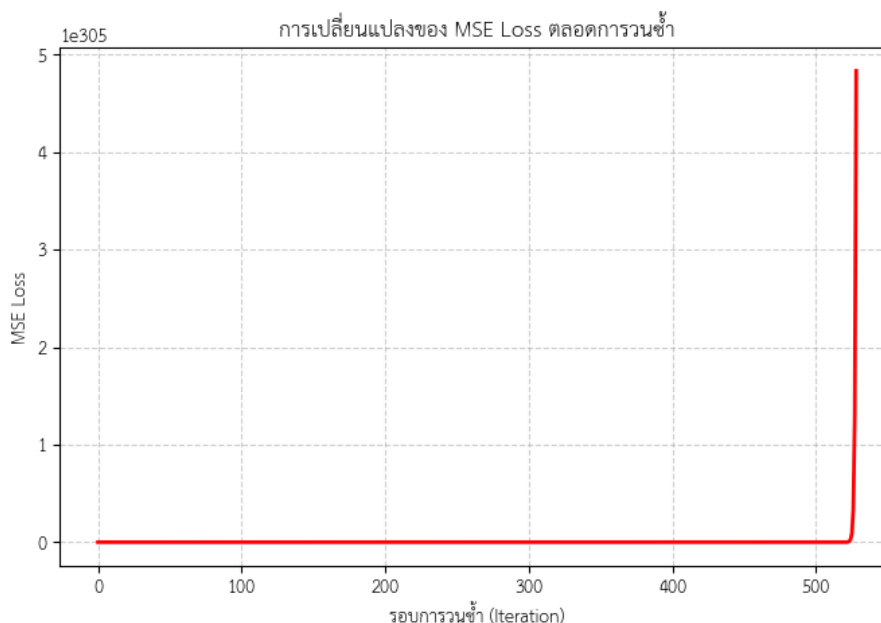
```

```
C1 = C1 - learning_rate * grad_C1
```

Task 5 — ผลลัพธ์และการแสดงผลภาพ (Results & Visualizations) และพิตกับข้อมูลได้ดีแค่ไหน

In [4]:

```
1 # กราฟที่ 1: การลดลงของ Loss (Learning Curve)
2 plt.figure(figsize=(8, 5))
3 plt.plot(range(n_iterations), history_loss, color='red', linewidth
4          =2)
5 plt.title('การเปลี่ยนแปลงของ MSE Loss ตลอดการวนซ้ำ')
6 plt.xlabel('รอบการวนซ้ำ (Iteration)')
7 plt.ylabel('MSE Loss')
8 plt.grid(True, linestyle='--', alpha=0.6)
9 plt.show()
```



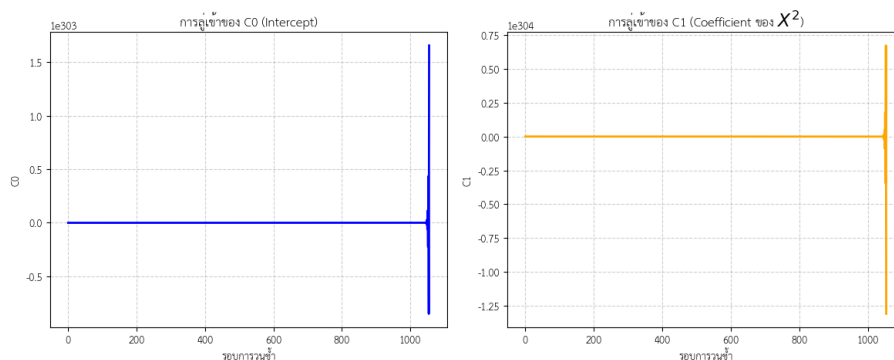
In [5]:

```
1 # กราฟที่ 2: เส้นทางการเข้าสู่ของพารามิเตอร์ C0 และ C1
2 plt.figure(figsize=(12, 5))
3
4 plt.subplot(1, 2, 1)
5 plt.plot(range(n_iterations), history_C0, color='blue', linewidth
6          =2)
```

```

6 plt.title('การรู้ค่าของ C0 (Intercept)')
7 plt.xlabel('รอบการวนซ้ำ')
8 plt.ylabel('C0')
9 plt.grid(True, linestyle='--', alpha=0.6)
10
11 plt.subplot(1, 2, 2)
12 plt.plot(range(n_iterations), history_C1, color='orange',
13          linewidth=2)
14 plt.title('การรู้ค่าของ C1 (Coefficient ของ $X^2$)')
15 plt.xlabel('รอบการวนซ้ำ')
16 plt.ylabel('C1')
17 plt.grid(True, linestyle='--', alpha=0.6)
18
19 plt.tight_layout()
20 plt.show()

```



In [6]:

```

1 # กราฟที่ 3: เส้นโค้งที่ฟิตแล้วเทียบกับจุดข้อมูลจริง (Fitted Curve)
2 plt.figure(figsize=(9, 6))
3
4 # จุดข้อมูลจริง
5 plt.scatter(X, Y, color='black', alpha=0.5, label='ข้อมูลจริง (Data)',
6            , edgecolor='k')
7
8 # สร้างจุด x สำหรับวาดเส้นโค้งให้เรียบเนียน
9 x_line = np.linspace(min(X), max(X), 100)
10 y_line = C0 + C1 * (x_line**2)
11
12 plt.plot(x_line, y_line, color='red', linewidth=3, label=f'โมเดล: $\\hat{y} = {C0:.4f} + {C1:.4f}x^2$')
13
14 plt.title('การฟิตโมเดล Quadratic เข้ากับข้อมูล')

```

```

14 plt.xlabel('X')
15 plt.ylabel('Y')
16 plt.legend(fontsize=12)
17 plt.grid(True, linestyle='--', alpha=0.6)
18 plt.show()

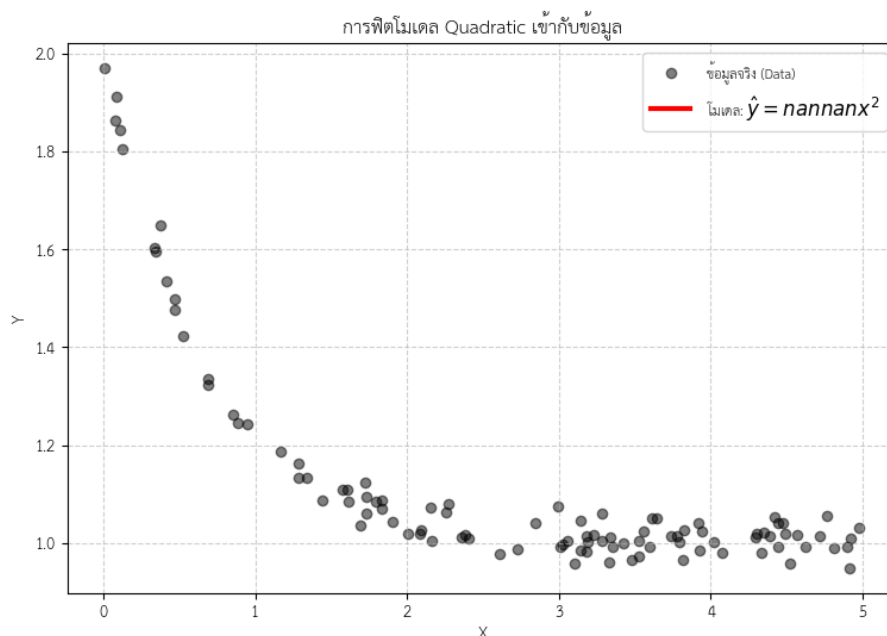
```

Out[6]:

```

<>:11: SyntaxWarning: invalid escape sequence '\h'
<>:11: SyntaxWarning: invalid escape sequence '\h'
C:\Users\Admin\AppData\Local\Temp\ipykernel_8024\3233163607.py:11:
  SyntaxWarning: invalid escap ...
  plt.plot(x_line , y_line , color='red' , linewidth=3 , label=f'โมเดล':
    $\hat{y} = {C0:.4f} {C1:. ...

```



In [7]:

```

1 # กราฟที่ 4: ความคลาดเคลื่อน (Residuals) แต่ละจุด
2 Y_pred_final = C0 + C1 * X_sq
3 residuals = Y - Y_pred_final
4
5 plt.figure(figsize=(9, 5))
6 plt.scatter(X, residuals , color='purple' , alpha=0.7, edgecolor='k'
7 )
8 plt.axhline(0, color='red' , linestyle='--' , linewidth=2)
9 plt.title('ความคลาดเคลื่อน (Residuals) แบ่งตามจุดข้อมูล X')
10 plt.xlabel('X')

```

```

10 plt.ylabel('Residual ($y_i - \hat{y}_i$)')
11 plt.grid(True, linestyle='--', alpha=0.6)
12 plt.show()

```

Out[7]:

```

<>:10: SyntaxWarning: invalid escape sequence '\h'
<>:10: SyntaxWarning: invalid escape sequence '\h'
C:\Users\Admin\AppData\Local\Temp\ipykernel_8024\3932470835.py:10:
  SyntaxWarning: invalid escap ...
  plt.ylabel('Residual ($y_i - \hat{y}_i$)')

```

